# Deadline Extractor

*Model: Gemma 4 e2b (5.1B, Q4_K_M) | Machine: AMD Ryzen 7 7435HS, 31.8 GB RAM, Windows 11 Home | Approach: Pipeline | https://youtu.be/18vh3nR060A*

## Problem Statement

University students and working professionals deal with important deadline information buried inside long documents every single day. A typical university student gets handed four or five course syllabuses, each one a dense PDF with assignment due dates, exam dates, lab submission windows, and participation deadlines scattered across five to fifteen pages all in different formatting. They also receive mid-semester emails from professors pushing dates around. Manually reading through all of that and copying every date into a calendar is slow, boring, and dangerously easy to get wrong. Miss one date and an assignment gets a zero.

The scoped problem this project addresses: given any combination of a PDF document, an image or screenshot, or raw pasted text, automatically identify every deadline or time-sensitive event, label it with a human-readable title, and convert the full list into a downloadable .ics calendar file that imports directly into Google Calendar, Apple Calendar, or Outlook without any manual re-entry.

## Persona

**Maya Chen -** *"I know all my deadlines are in those PDFs somewhere. I just never have time to dig them all out."*

| | |
|---|---|
| **BASIC PROFILE** | Age: 21 \| Year: Third-year \| Degree: Biomedical Engineering<br>Location: On-campus student, university Wi-Fi<br>Device: Personal Windows laptop (primary), phone for quick checks<br>Tech comfort: Comfortable but not technical - uses Notion, Google Drive, Discord, Canvas daily; has never written a script or automation |
| **USAGE CONTEXT** | Frequency: 4–6 times per semester (semester start + each time a lecturer emails a date change)<br>Choice vs. required: By choice - she actively looks for tools that reduce admin work<br>Trigger: Opening a new syllabus PDF and realising it is 22 pages long |

| | |
|---|---|
| **GOALS** | • Get every deadline from all five course syllabi into Google Calendar in one session at the start of term - not spread across five separate sittings.<br>• Stop relying on study-group Discord pings as her primary deadline reminder system. |

| | |
|---|---|
| | • Catch mid-semester date changes before they cause a missed submission, not after.<br>• Spend less than five minutes on calendar setup per course so she has time to actually read the material. |
| **FRUSTRATIONS** | • Each syllabus is a 15–25 page PDF. Deadlines are scattered: one in the week-by-week schedule, another in the assessment overview table, a third buried in a late-penalty footnote. She has to read the whole document to be sure she found them all - and she almost never does.<br>• She missed a lab report submission last semester because the due date was only mentioned in a footnote on page 19 of the course outline. She did not see it until the day after the deadline.<br>• When lecturers send 'date change' emails mid-semester, she has to find the original calendar entry, remember which course it belongs to, and update it manually. She often forgets the update step.<br>• Existing tools (Google Calendar, Notion, Canvas) do not read PDFs. She still has to do all the extraction by hand, which defeats the point of having a calendar app. |
| **BEHAVIORS** | • Downloads all syllabi on the first day of semester but rarely reads them fully until the night before something is due.<br>• Skims interfaces quickly - if important information is not visible in the first three seconds, she assumes it is not there.<br>• Prefers browser-based tools over installing software ('I don't want another app on this laptop').<br>• Will do a quick visual check of extracted results if the interface makes it feel fast - she is willing to spend 30 seconds scanning a list, not 5 minutes reading it.<br>• Abandons tools that require account creation before showing her any value. |
| **NEEDS & CONCERNS** | • Needs: A tool that accepts a PDF and returns a ready-to-import calendar file in under 30 seconds, with zero account setup.<br>• Needs: Enough context shown next to each extracted date (title, source snippet) that she can spot a wrong entry at a glance without re-reading the PDF.<br>• Concern: What if the tool misses a deadline? A false sense of completeness is worse than doing it manually, because she stops double-checking.<br>• Concern: She does not know what Ollama or a language model is and does not need to - but she will lose trust immediately if the tool gives her one obviously wrong date. |

**Direct connection to the problem statement:**

The problem statement identifies three specific pain points: deadlines buried across long PDFs, mid-semester email updates that are easy to miss, and the consequence of missing one date (a zero grade). Maya embodies all three. She receives 15–25 page syllabi she cannot fully read, she has already missed a deadline hidden in a footnote, and she has no reliable system for catching mid-semester date changes. Her technical comfort level is high enough to use a web tool without hand-holding, but low enough that she cannot build her own solution. She is exactly the person this tool is designed for - not because she is incompetent, but because the problem is genuinely tedious and the existing tools do not solve it.

## Scenario

It is the first Monday of semester. Maya has just received her four course outlines as PDF attachments. She opens the Deadline Extractor in her browser. The first thing she sees is the upload card. She drags her first syllabus PDF onto the PDF drop zone. The zone highlights briefly to confirm it received the file. She clicks the "Extract Deadlines" button.

The button changes to "Extracting..." while the system sends the file to the backend. The backend converts the PDF to markdown using pdfplumber, then sends that text to Gemma 4 e2b running locally through Ollama. For a document this long, extraction runs chunk by chunk and takes under a minute; the panel then switches to the results view. It shows nine deadlines, sorted by date.

Maya scans the list. Each row shows the date, an urgency badge (Overdue, Today, This Week, Upcoming), the title the model inferred, and a snippet of context from the original document. She notices one entry reads "Date: 2026-11-20" with no real title. She recognises that date as her final exam. The model missed the label. She makes a mental note. The other eight look right.

She clicks "Export as .ics calendar file." A file named deadlines.ics downloads instantly. She opens it. Google Calendar asks if she wants to import 9 events. She clicks yes. All nine show up in her calendar with the correct dates. She goes back to the Deadline Extractor and repeats for her remaining three syllabi. The whole process takes under five minutes. She used to spend forty-five minutes doing this by hand at the start of every semester, if she did it at all.

## Human-AI Interaction Design

**What the user does:**

- Drops a PDF, uploads a screenshot, or pastes text into the upload panel.
- Clicks one button to trigger extraction.
- Reads through the list of extracted deadlines and filters by urgency if needed.
- Clicks "Export as .ics calendar file" to download a ready-to-import calendar file.
- Clicks "← New input" to start over with another document.

**What the model does:**

Gemma 4 e2b receives document content as text through Ollama's chat API — PDFs are converted via pdfplumber, images via Tesseract OCR, and pasted text is passed through directly. The model

never processes images itself (see Model Limits Log: the vision pathway was non-functional and was removed). Its only job is to read that content and return a JSON array. Each item in the array must have four fields: title (a short human-readable label), date (strictly YYYY-MM-DD), time (HH:MM in 24-hour format, defaulting to 23:59 if none is specified), and description (a sentence of context). The model is told explicitly in the system prompt: return only the JSON array, no explanation, no code fences, no commentary.

**How the design handles model failures:**

- Code fence stripping: Despite the instruction, the model sometimes wraps its JSON in ```json ... ``` fences. The backend strips these with a regex before attempting to parse, so the response still works.
- JSON extraction fallback: If the response is not valid JSON at all, the backend runs a second regex looking for any [...] array anywhere in the string. If that also fails, it returns an empty list instead of crashing. The user sees zero deadlines and can retry.
- Deterministic urgency sorting: The model is not asked to judge whether a deadline is overdue or upcoming. That calculation is done in the frontend with plain JavaScript date arithmetic. This avoids the model making wrong relative-time judgements like treating a past date as upcoming.
- Mandatory human review: The extracted list is always shown to the user before export. The user can see the model's inferred title, the exact date, and a snippet of surrounding context. This gives the user a chance to catch hallucinations or missed labels before they land in their calendar.
- Low temperature: The model is called with temperature = 0.1. This makes its output more consistent across repeated calls with the same input and reduces creative hallucination.
- No fabrication of undeterminable dates: In testing, inputs with a deadline but no determinable date ("due end of term") consistently returned an empty result rather than an invented date. The residual gap — the user sees "no deadlines found" although an obligation exists — is addressed in the Plan.
- OCR routing for images: Screenshots never reach the model as pixels. Tesseract converts them to text first, and an OCR-quality gate checks the result — if the text is empty or unusable (e.g. handwriting), the LLM is skipped entirely and the user is told the image couldn't be read, with a manual-entry option, instead of a misleading "no deadlines found."
- Chunking for long documents: Document text is split into ~2000-character overlapping chunks, extracted per chunk, then merged and deduplicated. This keeps every model call inside the length envelope where JSON compliance and attention held in testing (verified 14/14 deadlines on a 20-page PDF).
- Source verification of dates: No date component reaches the UI unless it literally appears in the source text or is deterministically computed from something that does. If the model outputs a day number absent from the source, the date is discarded and recomputed from the text.
- Deterministic resolution of underspecified dates: When the source lacks a month or year, code — not the model — resolves the date (next occurrence of the stated day on or after today; document-level inference of slash-date convention, falling back to the local DD/MM convention).

- Visible inference badges: Any entry whose date is inferred rather than explicitly stated is marked "date estimated — confirm?" and can be edited before export. The system may make assumptions; it may not make them silently.

## Model Limits Log

| Input | What failed | Failure type & mechanism | Mitigation tried | Helped? (detail) |
|---|---|---|---|---|
| Partial dates with no year, submitted via text input: "Assignment 3 is due July 25" and "due March 12". | Both initially resolved to year 2024 — two years in the past. The UI flagged "Jul 25 2024" as 724 days overdue. Consistent across repeated runs (temperature 0.1). | Instruction-miss + hallucination. The prompt asked the model to "infer the most logical upcoming year," but never told it the current date — an LLM has no internal clock, so "upcoming" was unresolvable as written. With no anchor, the model defaulted to a training-data-era year (~2024). The instruction was impossible to follow, not merely ignored. | (1) Prompt/framing: injected today's date into the prompt. Partial — upcoming dates resolved correctly (July 25 → 2026), but passed dates didn't roll over (March 12 → 2026 "overdue" instead of 2027). (2) Fall back to non-AI: Python post-processing — if extracted date is past and source has no explicit year, add years until ≥ today. | Yes — March 12 → 2027, July 25 → 2026. Explicit-year check stops it altering genuinely past dates. The LLM finds date text reliably but can't do calendar arithmetic — that belongs in code. |
| Screenshots uploaded via the web interface returned no deadlines; to isolate the cause, a clean synthetic screenshot with explicit 'Due:' lines was sent directly to the model, bypassing the frontend | What failed: Extraction returned an empty list [] despite dates being clearly present. When asked to simply describe the image in plain text, the model produced completely unrelated content — a "grid of numbers", random years (2023/2024/500000) — different on every run. A second variant (gemma4:latest) instead claimed no image was provided. | Hallucination + inconsistency (vision input not grounding output): the output had no relationship to the image content at all, indicating the vision input was not conditioning generation in this local setup — not partial misreading, but a fully non-functional vision pathway. With no visual grounding, the model generates plausible-sounding content from its priors, and does so differently each run. | Fall back to non-AI: replaced the vision pathway entirely with Tesseract OCR. Screenshots are converted to text first, then routed through the same text-extraction pipeline already used for PDFs, so the LLM only handles text interpretation — the narrow task it is reliable at. | Yes — screenshots now route through the proven text pipeline; typed screenshots extract reliably (see later rows for OCR's own limit on handwriting) |
| Handwritten note (iPad, clear pink | No deadline extracted. OCR | Not an LLM failure — the non-AI | Narrow the task + human-in-the-loop. | Yes for trust, no for capability: the |

| | | | | |
|---|---|---|---|---|
| handwriting: "Machine Learning Homework 2, Due 28 July 2026") uploaded via the web interface, after the OCR fallback was added. | output was completely empty – 0 characters. | fallback has its own limit. Tesseract is trained on printed fonts and cannot segment cursive handwriting, so the LLM receives unusable text | (1) Scoped supported input to typed text and stated this in the UI ("Works best with typed text — handwriting isn't supported yet"). (2) Added an OCR-quality gate: if Tesseract returns empty or unusable text, the app skips the LLM entirely and tells the user the image couldn't be read, offering manual entry — instead of the misleading generic "no deadlines found." | handwritten note still can't be read, but the user now knows why and has a recovery path, rather than assuming the note contained no deadline. Skipping the LLM on garbage input also prevents it from hallucinating a date from garbled OCR text. |
| 20-page course-outline PDF, 14 deadlines with explicit full dates spread across all pages | Sent as one input, the model abandoned the required JSON format entirely — returning a markdown summary with headers and a table — and reported only the final 5 of 14 deadlines, itself stating it was working from "pages 15 through 20." The parser found no JSON → [], which triggered the day-only regex fallback on the full text, producing 14 plausible-but-wrong dates (correct titles/days, months replaced by next-occurrence logic). | Format-break + context-limit/forgetting, compounding: at ~20 pages, instruction-following from the system prompt collapses (JSON held on every short input tested) and attention retains only the document tail. The wrong dates shown to the user were then produced by a mitigation firing outside its design scope — failures cascade across layers. | (1) Chunking: split text into ~2000-char overlapping chunks, extract per chunk, merge + dedupe on title/date. (2) Made the regex fallback month-aware, so it only applies next-occurrence resolution when the source truly lacks a month. | Yes — 14/14 deadlines, all dates correct, JSON compliance restored in all 20 chunks (overlap duplicates removed by dedupe). Cost: 20 sequential model calls (~1 min total) instead of one. Lesson: keep each call inside the envelope where the model behaves; and constrain fallbacks to their design scope, or they convert one failure into another. |
| Day-only dates, no month/year, pasted text: "PDS project due on 4th" "Algorithm Homework due on 12th ", "SDS project | Model fabricated the missing month with no consistent strategy across same-structure inputs: "4th" → 2026-07-04 (past, | Hallucination + inconsistency. The strict YYYY-MM-DD schema forces a complete date, so the model confabulates the | Fall back to non-AI + human-in-the-loop: post-processing checks the source text for an explicit month; if absent, code discards the | Yes — all three inputs now resolve deterministically: 4th → 2026-08-04, 12th → 2026-08-12, 27th → 2026-07-27, every run. Residual: "next |

| | | | | |
|---|---|---|---|---|
| due 27th". Today: 19 July 2026. | shown overdue), "12th" → 2027-07-12 (a full year out), "27th" → 2026-07-27 (correct only because the day falls after today). | missing month — arriving at different resolutions for equivalent inputs, even at temperature 0.1. The uncertainty flag comes from the same unreliable generation process, so the model cannot audit its own guessing. | model's month and computes the next occurrence of that day ≥ today (day already passed → next month; not yet passed → this month). inferred is set in code, and flagged entries get a "date estimated — confirm?" badge before export. | occurrence" is a chosen convention, not extracted fact — hence the confirmation badge rather than silent export. |
| Phrasing grid of the same deadline, two baseline rounds (no changes between): "PDS due 9", "PDS due 9th", "PDS Project due 9", "PDS Project due 9th", "PDS due 9 Aug", "PDS Project due 9 Aug". | Day-only forms extracted non-deterministically: across rounds "PDS due 9th" flipped fail→pass and "PDS Project due 9th" flipped pass→fail on identical input; bare "PDS due 9" failed all runs. Forms with an explicit month passed every run. | Inconsistency (brittleness): day-only phrasings sit on an unstable recognition boundary that flips between runs even at temperature 0.1 — it cannot be mapped or prompted around. An explicit month is a strong enough signal to clear the threshold reliably. | Fall back to non-AI — regex safety net: when the model returns [] but the text matches a due/submit/deadline + day pattern, code extracts the day, resolves it with the next-occurrence logic, and presents it as an estimated date requiring confirmation. Model hits route through the same date resolution, so both paths yield identical results. (Prompt-level few-shot fixes not pursued: baseline instability made prompt fixes unpromising, and the deterministic net covers all phrasings.) | Yes — "PDS due 9" and all grid inputs now produce a consistent, badged result regardless of whether the model's extraction fires; the model's coin-flip no longer affects what the user sees. Residual: the regex covers common due/submit/deadline phrasings; unusual wordings fall through to manual entry. |
| Ambiguous slash date, pasted text, 6 runs: "Assignment due: 07/08/2026" (July 8 in MM/DD; August 7 in DD/MM). Control: "due: 25/08/2026" (unambiguous — only DD/MM valid), 2 runs. | The model resolved the ambiguity inconsistently and silently: 4/6 runs → 2026-07-08 (MM/DD), 2/6 → 2026-08-07 (DD/MM), with no flag or note either way — the same text produces deadlines a month apart depending on the run. Control: 25/08 → 25 Aug in both runs, so the | Inconsistency + instruction-miss (unsurfaced ambiguity). When both readings are valid, the model neither commits to one convention nor signals the ambiguity — it leans MM/DD (training-data prior) but flips between runs. For our users (Singapore, DD/MM convention), the | Fall back to non-AI + human-in-the-loop: regex detects slash dates where both leading numbers ≤ 12; these bypass the model's interpretation — parsed as DD/MM per user locale, flagged inferred, badged "format assumed DD/MM — confirm?". Unambiguous slash dates parsed | Yes — 07/08/2026 now resolves to 7 Aug 2026 with the "format assumed DD/MM — confirm?" badge on every run; the model's interpretation no longer affects the result. 25/08 parses as 25 Aug, unbadged. Residual: DD/MM is a locale assumption, not extracted fact — |

| | | | | |
|---|---|---|---|---|
| | failure is specific to ambiguity resolution, not slash-date parsing. | majority MM/DD readings are silently wrong for the likely intent. | deterministically in code, no badge. Follow-up (first real-world input): a real LMS screenshot used MM/DD — "7/22/26" (only valid as MM/DD) alongside ambiguous "8/12/26" — and the DD/MM assumption mis-parsed the latter as 8 December; the confirm badge surfaced the error. Extended the mitigation with document-level inference: unambiguous slash dates in a document determine the convention for all its slash dates, falling back to the locale assumption (badged) only when no unambiguous date exists. Re-test: same screenshot now yields Jul 22 and Aug 12 correctly. | hence confirm-before-export; a future version could make the convention a user setting. |
| Realistic email-style text: "reminder that the PDS project must be submitted by the 9th. Late submissions lose 10%." | Model output date 2026-07-28 — day 28 appears nowhere in the source; the explicit "9th" was ignored and replaced (raw output, 02:37:40). Post-processing trusted the model's day and displayed "Jul 28, date estimated." | Hallucination — but replacing present information rather than filling missing information: the most dangerous variant for extraction, since output looks as plausible as a correct one. Also exposed an assumption in the earlier mitigation, which verified the month but trusted the day. | Verification step — extracted day must literally appear in the source text (as number/ordinal near date context); otherwise the date is recomputed from the text via the regex path, or routed to manual entry. Principle: no date component reaches the UI unless verbatim from source or deterministically derived from something that is. | Yes — same email now resolves to Aug 9 (badged) on every run; typing "9" can no longer surface as 28. Residual: verification covers the day number; a wrong-but-present day (text mentions two numbers, model picks the wrong one) would still pass the check — noted for future evaluation. |

## Reflections

### 1. User Needs

Maya's primary need is time savings with acceptable accuracy. She does not need a perfect tool. She needs a tool that is fast, that gives her most of the deadlines correctly, and that lets her quickly catch the ones it gets wrong before they matter. Secondary needs are trust (she has to believe the output is worth checking) and zero setup friction (no account, no installation, no config).

### 2. Why This Design Meets Those Needs

The pipeline architecture keeps the AI responsible for only the genuinely hard part: understanding natural language and extracting structured date information from unstructured text. Everything else in the pipeline is deterministic and reliable. pdfplumber does structured PDF-to-text conversion with no AI involved. The frontend does urgency sorting with plain arithmetic. The icalendar library generates the .ics file from a Python dictionary with no AI involved. This means the system's failure surface is limited mostly to the LLM step, and the human review panel is placed exactly where the risk is highest, right after that step.

The single-button interaction matches Maya's expectation that things work immediately. There is no configuration screen, no settings, no account. Upload, click, review, export. The urgency colour-coding means Maya can glance at the list and immediately see if anything is this week without reading every row.

### 3. How Well It Works

For clean, well-formatted text, the system performs well. Informal observation across development testing (synthetic documents, pasted emails, and one real LMS screenshot) suggests it correctly extracts most deadlines on the first try — roughly 85 to 90 percent — with most failures being missed dates rather than wrong dates. Wrong dates (hallucinations) are rarer but more dangerous, which is exactly why the review step exists.

Performance degrades significantly for very long documents (over 25 pages), scanned or handwritten images with small text, and documents with ambiguous date formats or relative language. These are real limits, not edge cases, for the student use case.

### 4. What Else Could Work

- Rule-based regex extraction: Fast, zero hallucination, zero cost. But it misses natural-language dates like 'two weeks from submission' and requires manual format rules for every date style.
- A fine-tuned model dedicated to date extraction: More reliable than a general-purpose chat model for this narrow task. Requires a labelled training dataset of documents with ground-truth date annotations.
- A retrieval-augmented approach: Pre-scan the document with regex to find candidate date strings, then send only those strings plus their surrounding context to the LLM. This would reduce the model's input length and focus it on the hard cases only.

- Browser extension: Instead of a separate app, inject the extraction tool directly into the PDF viewer or email client. Lower friction for users who already have the document open.

### 5. What Gemma 4's Limits Forced the Design to Work Around

The most consequential constraint was the context window. On a 20-page test PDF containing 14 known deadlines, the model sent the full document abandoned its output format entirely and reported only the final five entries — stating it was working from "pages 15 through 20" — with no signal that it had truncated (see Model Limits Log). This forced a chunking architecture: text is split into overlapping ~2000-character chunks, extracted per chunk, merged, and deduplicated. Re-testing the same document recovered all 14 deadlines with correct dates and full JSON compliance in every chunk. The residual cost is latency (about twenty sequential model calls, roughly a minute) and the honest caveat that documents far beyond 20 pages remain untested.

Temperature is fixed at 0.1 so that repeated runs on the same input are stable. Even at this setting, the model remained brittle across phrasings and occasionally across runs (see Model Limits Log), which is why the deterministic fallbacks and verification steps exist — low temperature reduces randomness but does not produce reliability.

The model's tendency to ignore output format instructions forced two layers of defensive parsing in the backend: a code-fence stripper and a fallback JSON array extractor. Without these, a single model response with an extra sentence before the JSON would crash the entire extraction.

The model's hallucination on ambiguous inputs forced the design to prioritise the human review step as a non-optional part of the flow. The results panel cannot be skipped. The user always sees the list before downloading. This was not the original design, which had an option to auto-download. That was removed specifically because the model is not reliable enough to skip human eyes.

## Report Constraints & What Could Be Improved

This report has real limits that should be named honestly rather than papered over.

**Constraints on the current report:**

- No formal evaluation data. The 85-90% accuracy figure quoted in Reflections is an informal estimate from watching test runs, not a measured precision/recall score against a labelled ground-truth set. It could be significantly off in either direction.
- Persona is hypothetical. Maya Chen was constructed from general knowledge of student behaviour, not from user interviews or surveys. Her actual needs and frustrations may differ from what is described here.
- Small failure sample. The model limits log covers 8 failure types observed during development. A wider test across more document types, languages, and formats would likely reveal additional failure modes not captured here.
- Binary helped/not-helped framing understates nuance. Even where a mitigation is marked as working, it often introduced a trade-off (e.g. fixing hallucination by omitting entries also drops

legitimate vague deadlines). The log now describes these trade-offs but a fuller account would quantify them.

- No latency data. Response time varies by document length and machine load but no systematic measurements were taken. The claim that extraction takes 'two to four seconds' is observational and not benchmarked.

**What could be improved in the report:**

- Run a benchmark. Test 15-20 real syllabi with manually annotated dates and report actual precision (how many extracted dates were correct) and recall (how many real dates were found). This replaces the informal estimate with a defensible number.
- Validate the persona through at least three short interviews with real students. Check whether the stated needs, friction points, and willingness to do a review step actually hold.
- Expand the limits log with longer-tail failures: non-English documents, PDFs with scanned image pages (no text layer), heavily formatted academic papers with footnotes containing dates, and recurring events (every Friday) that the model should either expand or flag.
- Add a scenario where the tool fails visibly and describe what the user experience looks like. The current scenario only covers the happy path. A failure scenario would show how the design degrades gracefully rather than just how it works when everything goes right.

## Plan

### Mitigations for Model Limits

- Context window: Chunking is implemented and verified (14/14 on a 20-page document). Next: split on section boundaries instead of fixed character counts to avoid cutting a deadline sentence in half, and parallelise chunk calls to reduce the ~1 minute latency on long documents.
- Inconsistency: Run the model twice on high-stakes documents and take the union of both result sets after deduplication. Show the user a confidence indicator (one result from both calls = high confidence, only one call = lower).
- Hallucination on ambiguous dates: Extend the existing regex safety net (currently a rescue path when the model returns nothing) into a pre-pass: identify candidate date strings first and send the model only those strings plus ~80 characters of context. This narrows the task and gives the model much less room to invent.
- Format-break on structured content: Already partly addressed with pdfplumber's table extraction. Next step: provide the model with examples of table rows in the few-shot section of the system prompt so it learns the expected structure from context.

### AI + Non-AI Methods

The current pipeline already combines both. pdfplumber (non-AI) handles the PDF layer. Gemma 4 e2b (AI) handles the language layer. icalendar (non-AI) handles the output layer. The plan extends this:

- Add regex pre-screening (non-AI) as a first pass to anchor the model's task.

- Add a confidence score display based on how many model runs agreed on each deadline.
- Keep the export pipeline fully deterministic. The AI never touches the .ics file directly.

### What to Evaluate Next

- Precision and recall on a benchmark set of 15 real course syllabi with manually annotated ground-truth deadlines. This gives a concrete accuracy number.
- User study with 5-8 students. Measure: time from opening the tool to all deadlines in calendar, number of errors the review step caught, number of errors that slipped through into the calendar, and subjective confidence in the tool.
- Track how often the human review step actually leads to a user spotting and discarding a deadline. This tells us the real error rate in production and whether the review step is worth the friction it adds.
- Latency benchmarks across document sizes. Identify the document length at which response time becomes unacceptable to users (hypothesis: over 10 seconds).

## AI-Use Acknowledgment

The following AI tools were used in this project. All use is disclosed in accordance with the course plagiarism policy.

**Gemma 4 e2b (Google, accessed via Ollama):**

This is the primary AI model in the system. It runs at inference time to extract deadline information from document content. It is not used during development, report writing, or any other part of the project outside of its designated role in the extraction pipeline.

**Claude (Anthropic, accessed via Claude Code):**

Claude Code was used as a coding assistant throughout development: it wrote and debugged the FastAPI backend, React frontend, and the failure mitigations described in the Model Limits Log, and it executed some diagnostic tests during debugging (notably the vision-pathway isolation tests), whose raw outputs the author reviewed in the session transcript and in the pipeline's output log.

Claude (claude.ai) additionally helped plan the testing methodology (which failure cases to probe and how to verify them), generated synthetic test documents used as inputs (the vague-deadline PDF and the 20-page course-outline PDF with its answer key), helped draft the wording of the Model Limits Log rows and this report from the author's recorded test results, and helped script the demo video.

All test inputs, observed outputs, and log contents come from tests the author ran or directly observed; every log row is backed by timestamped raw model outputs in the project's log file. Claude assisted with structure and wording, not with the observations themselves.